

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of	20%	Concepts are	Mostly accurate	Partial application	Weak application

Concepts		applied accurately to practical tasks	application		
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments — Questions & Answers

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED: CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Assessment Tool Weightage: 40%

Bloom's Taxonomy: Precision (Level 3)

Question Count: 5

Total Marks: 40

Experiment 1

Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.

ANSWER

Objective: To compare object, block, and file storage models and select the most suitable model for backups, VM disks, and shared enterprise documents.

Architecture Sketch

Client / Application → Storage Gateway / Driver → three parallel storage backends: (a) Object Store [REST API, bucket/key], (b) Block Store [SAN/iSCSI LUN mounted to VM], (c) File Store [NFS/SMB share mounted as a network drive].

Comparison Table

Aspect	Object Storage	Block Storage	File Storage
Access unit	Object (data + metadata + key)	Fixed-size blocks	Files in folder hierarchy
Access method	REST/HTTP API	iSCSI / Fibre Channel	SMB / NFS
Best fit	Backups, images, unstructured data	VM boot/data disks, databases	Shared departmental documents
Scalability	Virtually unlimited, flat namespace	Limited by volume/LUN size	Limited by file-system/NAS capacity
Latency	Higher (HTTP overhead)	Lowest (raw I/O)	Moderate

Recommended Mapping

- Backups → Object storage (durable, cheap, versioned, scales to petabytes).
- VM disks → Block storage (low-latency raw I/O required for OS/boot performance).
- Shared enterprise documents → File storage (familiar folder access, concurrent multi-user editing via SMB/NFS).

Procedure

- Provision one bucket (object), one volume/LUN (block), one file share (file) on the chosen cloud platform.
- Upload/attach a sample workload to each: an image set to the bucket, an OS image to the block volume, shared docs to the file share.

- Measure and record latency, throughput, and access method for each.
- Tabulate results against the comparison criteria above.

Expected Output

A completed comparison table (as above) backed by measured latency/throughput figures, with a clear justification statement for each of the three use cases (backup, VM disk, shared documents).

Conclusion

No single storage model is universally best — the correct choice depends on the access pattern. Object storage wins on scale and cost for unstructured backups, block storage wins on raw performance for VM disks, and file storage wins on ease of shared, hierarchical access for enterprise documents.

Experiment 2

Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.

ANSWER

Objective: To develop and document a REST API workflow covering request, response, authentication, and deployment for a student portal (e.g., viewing grades/attendance).

Architecture Sketch

Browser/Mobile Client → HTTPS → API Gateway → Authentication Service (issues JWT/OAuth token) → REST API Service (Node.js/Spring Boot) → Database (student records) → Response returned as JSON back through the same path.

Procedure

- Design endpoints, e.g. POST /api/login, GET /api/students/{id}/grades, GET /api/students/{id}/attendance.
- Authentication: client sends credentials to POST /api/login; server validates and returns a signed JWT.
- Client stores the JWT and attaches it as an Authorization: Bearer <token> header on subsequent requests.
- Server middleware validates the token before routing the request to the business logic layer.
- Business logic queries the database, formats a JSON response, and returns an HTTP status code (200 OK, 401 Unauthorized, 404 Not Found, etc.).
- Deploy the API as a container/App Service behind the API Gateway; enable HTTPS, rate limiting, and logging.

Sample Request / Response

Step	Detail
Request	GET /api/students/1042/grades Header: Authorization: Bearer <JWT>
Response	200 OK Body: { "studentId":1042, "subjects": [{"code":"CSA15","grade":"A"}] }
Auth failure	401 Unauthorized Body: { "error":"Invalid or expired token" }
Deployment	API packaged in a container, deployed to a managed app service, exposed via API Gateway with HTTPS + auto-scaling

Expected Output

A working (or documented) end-to-end call: successful login returning a token, an authenticated GET request returning correct JSON grade data, and a rejected call without a valid token.

Conclusion

The experiment demonstrates the standard REST pattern — stateless requests, token-based authentication, JSON payloads, and clear HTTP status codes — which forms the backbone of nearly all cloud-native web applications.

Experiment 3

Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.

ANSWER

Objective: To design a serverless (event-driven) flow that processes an uploaded image using object storage, Function-as-a-Service (FaaS), and a notification service.

Architecture Sketch

User uploads image → Object Storage Bucket (raw/) → Bucket "object created" event triggers → FaaS Function (resize/thumbnail/virus-scan/tag) → processed file written to Object Storage (processed/) → FaaS publishes a message → Notification Service (email/SMS/push or message queue) → User or downstream service notified.

Procedure

- Create two storage prefixes/buckets: raw/ for uploads and processed/ for outputs.
- Configure an event trigger so any new object in raw/ invokes the FaaS function automatically.
- Function logic: read the image, generate a thumbnail/apply validation, write the result to processed/.
- On success, the function publishes a completion event to a notification/queue service.
- Notification service sends a confirmation (email/push) or forwards the event to another downstream consumer.
- Add error handling: failed processing routes to a dead-letter queue for retry/inspection.

Expected Output

An uploaded test image automatically produces a processed/resized copy within seconds, with a corresponding notification delivered — all without any server being manually provisioned or managed.

Conclusion

The event-driven, serverless pattern (storage trigger → function → notification) scales automatically with upload volume, incurs cost only during actual execution, and removes the operational burden of managing servers — making it well suited to bursty, unpredictable workloads like user uploads.

Experiment 4

Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.

ANSWER

Objective: To compare edge and fog computing architectures for a smart traffic management system, covering architecture, latency, and deployment location.

Architecture Sketch

Layer 1 — Edge: traffic cameras/sensors with on-device inference (vehicle/pedestrian detection) at each junction.
 Layer 2 — Fog: a regional fog node/gateway aggregating data from several junctions, performing short-term signal-timing optimization. Layer 3 — Cloud: central platform for city-wide analytics, historical trend modeling, and long-term planning.

Comparison Table

Aspect	Edge Computing	Fog Computing
Deployment location	At/near the sensor (junction camera, roadside unit)	Regional gateway/node serving multiple edge points
Latency	Lowest (milliseconds) — used for instant signal changes	Low-to-moderate — used for cross-junction coordination
Scope of data	Single sensor/junction	Aggregated across a district/zone
Typical task	Detect vehicle presence, trigger immediate signal change	Optimize green-wave timing across several junctions
Link to cloud	Sends summarized events upward	Forwards aggregated analytics to the cloud

Procedure

- Deploy lightweight inference at each junction (edge) to detect vehicle queues in real time.
- Deploy a fog node per zone that collects edge events from multiple junctions and computes coordinated signal timing.
- Forward zone-level summaries to the cloud for city-wide dashboards and historical analytics.
- Measure round-trip latency at each layer to confirm edge < fog < cloud.

Expected Output

Measured latency confirms edge-level signal decisions complete in milliseconds, fog-level coordination completes in a small number of seconds, and cloud analytics run on a longer, non-real-time cycle.

Conclusion

A layered edge–fog–cloud architecture gives smart traffic systems both instant local reaction (edge) and coordinated regional optimization (fog), while still feeding the cloud for citywide, long-horizon analytics — no single layer alone can satisfy all three needs.

Experiment 5

Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

ANSWER

Objective: To demonstrate a full cloud application deployment workflow: client access, browser interaction, web API call, storage access, and monitoring.

Architecture Sketch

End User → Browser (client-side app) → HTTPS request → Load Balancer → Web API Service (application logic) → Storage/Database layer (reads/writes data) → Response returned to browser → Monitoring & Logging service observes every hop (requests, errors, latency, resource usage).

Procedure

- Package the application (frontend + API) and deploy it to a cloud platform (App Service/containers).
- Configure a load balancer / gateway in front of the API for scalability and HTTPS termination.
- User opens the browser, the client app loads and issues an API call (e.g., GET /api/data).
- API service authenticates the request and reads/writes to the storage layer (database or object storage).
- Response flows back to the browser and renders in the UI.
- Enable monitoring/logging (metrics, traces, alerts) across every component to track health and performance.

Expected Output

A deployed application accessible via a public URL, where a browser action successfully triggers an API call, retrieves/stores data, and the monitoring dashboard shows the corresponding request log, latency, and status code.

Conclusion

This experiment ties together every CO3 component — browser client, web API, storage — into one deployable pipeline, and shows why monitoring at each hop is essential for diagnosing failures and maintaining reliability in production cloud applications.